PROTOCOL AWARE MACHINE LEARNING FRAMEWORK FOR INTELLIGENT MODBUS ATTACK DETECTION AND INCIDENT ANALYSIS

Command and Response Injection Detection in Modbus/TCP Using Context-Aware Command Analysis

Project ID: J26-CS-318

PROJECT PROPOSAL REPORT

Rathnayaka K.H. - IT23150270

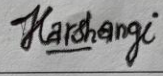
B.Sc. (Hons) Degree in Information Technology
Specializing in Cyber Security

Department of Computer Systems Engineering
Sri Lanka Institute of Information Technology, Sri Lanka

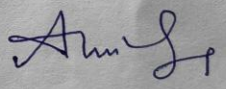
September 2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Student	Student ID	Signature
Rathnayaka K.H.	IT23150270	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Role	Name	Signature
Supervisor	Amila Senarathna	

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Student	Student ID	Signature
Rathnayaka K.H.	IT23150270	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Amila Senarathna	15/09/2026	

ABSTRACT

Modbus/TCP is still popular in industrial controls and supervisory applications but does not offer some of the built-in integrity and/or authentication protections found in other primary protocols. This allows an opponent to send a valid command or a spoofed reply that might look like a typical command or reply if seen in isolation. Intrusion detection has been studied by existing systems using signatures, generic flow statistics, individual protocol fields, or sequence analysis, individually. Very few works combine the reconstructed request-response relationships with operational context, in an explainable model, to differentiate between normal traffic, command injection and response injection.

The purpose of this research is to propose a Modbus/TCP context-aware detection component. It makes ordered traffic into request-response events and engineers' protocol-semantic features to describe function-code transitions, read/write ordering, register-access behavior, timing, response consistency, source-role behavior (where available) and value transitions. Each contextual event will then be classified as normal, command injection or response injection using a supervised Random Forest classifier. The main experiment will compare the context-aware Random Forest with an algorithm trained only on isolated packet/static features to measure the contribution of context without being intertwined with the classifier. XGBoost can be taken as an additional benchmark.

The development of the models will be primarily based on the CICModbusDataset2023, which contains benign and attack Modbus/TCP PCAP captures with attack logs and timestamps. Time, target IP, attack scenario and transaction identifier (if available) will be used to correlate packet records with the logs. Then the direction and request-response relationships will be checked to verify them, prior to candidate false-data-injection events being mapped to command or response injection. Secondary validation will be provided by the MSU New Gas Pipeline dataset that explicitly includes command- and response-injection categories. There will be no leakage in the data by splitting them by scenarios, capture, or time. Macro precision, macro recall, macro F1 score, false positive rate, ROC-AUC, confusion matrix, detection latency, and per-class recall will be used to assess performance.

ACKNOWLEDGEMENT

I acknowledge the guidance of my supervisor and the support of the research team. I also acknowledge the researchers and institutions that make industrial-control datasets and technical publications available for academic study. Final names and institutional acknowledgements will be added after confirmation.

Table of Contents

DECLARATION 3
ABSTRACT 4
ACKNOWLEDGEMENT 5
LIST OF TABLES 9
LIST OF FIGURES 10
LIST OF ABBREVIATIONS 11
LIST OF APPENDICES 12
1. INTRODUCTION 13
1.1 Background and Context 13
1.2 Research Problem 14
1.2.1 Problem Scope and Boundaries 14
1.3 Literature review 15
1.3.1 Modbus/TCP Security and Command Injection 15
1.3.2 Command and Response Injection Detection 15
1.3.3 Model-Based Modbus/TCP Intrusion Detection 16
1.3.4 Machine Learning for Malicious SCADA Communication 16
1.3.5 Industrial-Control Traffic Datasets 16
1.3.6 CICModbusDataset2023 16
1.3.7 Random Forest for the Proposed Component 17
1.3.8 Critical Synthesis of the Literature 17
1.3.9 Existing Systems and Products 17
1.3.10 Limitations Addressed by the Proposed Component 18
1.4 Research Gap 19
2. OBJECTIVES 21
2.1 Main Objective 21
2.2 Specific Objectives 21
3. RESEARCH DESIGN AND METHODOLOGY 22
3.1 Research Design 22
3.2 Data Availability and Preparation 22
3.2.1 CICModbusDataset2023 Labelling Process 23
3.2.2 Secondary MSU Label Mapping 24
3.2.3 Data-Quality Assessment 25
3.2.4 Preprocessing Tools 25
3.3 Contextual Feature Set 25

3.3.1 Feature Construction 26
3.3.2 Prevention of Target Leakage 26
3.4 Model Selection: Why Random Forest? 26
3.4.1 How Random Forest Works in Command and Response Injection Detection in Modbus/TCP Using Context-Aware Command Analysis 27
3.4.2 Candidate Training Configuration 27
3.5 Baseline, Training, and Validation 28
3.5.1 Experimental Matrix 28
3.5.2 Evaluation Interpretation 29
3.5.3 Explainability and Error Analysis 29
3.6 Data Security and Ethics 29
3.6.1 Reproducibility and Governance 29
3.6.2 Threats to Validity 29
3.7 Development Approach 30
4. HIGH-LEVEL SYSTEM ARCHITECTURE 31
4.1 System Overview 31
4.1.1 Component Inputs and Outputs 31
4.1.2 Runtime Behavior 31
4.2 Required Technical Figures 32
4.3 Command and Response Injection Detection Processing Pipeline 32
4.4 Overall Integration and Dependencies 33
4.4.1 Proposed Result Contract 33
4.5 Individual Responsibility research Boundary 34
5. USER REQUIREMENTS 36
5.1 Stakeholders and Elicitation Plan 36
5.1.1 Planned Evidence Collection 36
5.2 Functional Requirements 36
5.3 Non-Functional Requirements 37
5.4 Acceptance Test Summary 37
6. COMMERCIALIZATION PLAN 38
6.1 Target Market, Customer Persona, and Key Stakeholders 38
6.2 Value Proposition 38
6.3 Revenue Model 39
6.4 Cost Estimation 40
6.5 Pricing Strategy 40
6.6 Maintenance, Versioning, and Patch Updates 41

6.7 Competitive Advantage 42
6.8 Intellectual Property Considerations 42
7. BUDGET AND JUSTIFICATION 44
7.1 Resource and Project Risk Justification 44
7.2 Project Risk Register 45
8. WORK BREAKDOWN STRUCTURE (WBS) 46
9. GANTT CHART 47
10. REFERENCES 48
11. APPENDICES 50
Appendix 1 - Preliminary Dataset and Experimental Plan 50
Appendix 2 – Survey Evidence 50
Appendix 3 - Mandatory Proposal Evidence and Decision Log 51
Appendix 4 - AI Use Disclosure 52

LIST OF TABLES

Table 2: Data Availability and Preparation....22
Table 3: CICModbusDataset2023 Labelling Process....24
Table 4: Secondary MSU Label Mapping....24
Table 5: Contextual Feature Set....26
Table 6: Candidate Training Configuration....27
Table 7: Experimental Matrix....28
Table 8: Threats to Validity....30
Table 9: Component Inputs and Outputs....31
Table 10: Proposed Result Contract....34
Table 11: Individual Responsibility Boundary....35
Table 13: Project Risk Register....45
Table 14: Work Breakdown Structure....46

LIST OF FIGURES

Figure 1: High-Level System Architecture 32
Figure 2: Processing pipeline for context-aware Modbus/TCP command and response injection detection 33
Figure 3: Gantt Chart 47

LIST OF ABBREVIATIONS

CIC - Canadian Institute for Cybersecurity

CSV - Comma-Separated Values

HMI – Human-Machine Interface

ICS - Industrial Control System

IDS - Intrusion Detection System

ML - Machine Learning

OT - Operational Technology

PCAP - Packet Capture

PLC - Programmable Logic Controller

RF - Random Forest

ROC-AUC - Area Under the Receiver Operating Characteristic Curve

SCADA - Supervisory Control and Data Acquisition

TCP - Transmission Control Protocol

LIST OF APPENDICES

Appendix 1 - Preliminary dataset and experimental plan

Appendix 2 - Survey Evidence

Appendix 3 - Mandatory proposal evidence and decision log

Appendix 4 - AI use disclosure

1. INTRODUCTION

1.1 Background and Context

Industrial Control Systems (ICS) are computer-based systems that monitor and control critical infrastructure and industrial processes. They are used in power generation and distribution, water and wastewater treatment, oil and gas pipelines, food processing, and manufacturing. These environments contain supervisory applications, Human-Machine Interfaces (HMIs), Programmable Logic Controllers (PLCs), Remote Terminal Units (RTUs), sensors, and actuators that exchange operational commands and measurements.

Modbus is one of the communication protocols used within these environments. Modbus/TCP enables Modbus communication over TCP/IP networks and supports operations such as reading measurements, controlling coils, and modifying register values. However, Modbus was originally designed for simple and reliable industrial communication rather than modern cybersecurity requirements. It does not provide strong native authentication, encryption, or cryptographic message-integrity protection [1], [2]. Consequently, a receiving device may be unable to verify whether a message originated from an authorized source or was modified during transmission.

Historically, many industrial networks were physically isolated from conventional information-technology networks. Modern industrial organizations increasingly require remote monitoring, centralized administration, data sharing, and integration between IT and OT environments. Although this connectivity improves operational efficiency, it also increases the possible paths through which an attacker may reach industrial devices [2], [4], [11].

Attackers can exploit Modbus/TCP environments in several ways. A denial-of-service attack may overwhelm a system with excessive traffic and affect availability. Reconnaissance may be used to identify active devices, accessible addresses, and supported operations. Command injection inserts an unauthorized or falsified control request, while response injection creates or modifies a reply to misrepresent the state or result of an operation [6]. A replay attack captures a previously valid transaction and retransmits it later.

The complete group solution therefore consists of a protocol-aware detection framework with four specialized components:

- Denial-of-service detection
- Reconnaissance detection
- Command and response injection detection
- Replay-attack detection

These components examine different attack behaviors and send their results to a shared decision-fusion and incident-analysis layer. The integrated framework then produces correlated alerts, confidence information, and incident explanations.

The proposed Command and Response Injection Detection component focuses specifically on malicious commands and responses that may appear structurally valid. An injected message may contain an accepted function code, correctly formatted fields, and a legitimate-looking register address while still being operationally inappropriate. For example, a valid write command may be issued by an

unexpected source, target a restricted register, produce an unusual value change, or conflict with the preceding command sequence.

Existing security approaches often depend on fixed rules, protocol-validity checks, isolated-packet inspection, or general network-flow measurements. These methods can detect known signatures, malformed messages, and high-volume anomalies. However, they may not sufficiently represent Modbus command meaning, request-response relationships, source roles, register-access behavior, or surrounding operational context [3], [6], [8]. As a result, a protocol-valid command or response may remain undetected when its abnormality becomes visible only through contextual analysis.

The proposed component addresses this problem by reconstructing Modbus request-response events and engineering contextual features from command order, access permissions, timing, pair consistency, source behavior, and value transitions. A supervised Random Forest classifier then uses these features to classify events as normal, command injection, or response injection. The component also produces a confidence score and explanatory evidence for integration with the complete framework.

This research primarily supports United Nations Sustainable Development Goal 9-Industry, Innovation and Infrastructure-by contributing to the security and resilience of industrial infrastructure. It also has a secondary relationship with Sustainable Development Goal 7-Affordable and Clean Energy-because Modbus/TCP is used in energy and utility environments. The project aligns with the AIMS research group and the Cybersecurity and Information Assurance research cluster. It also relates to Artificial Intelligence and Machine Learning using machine-learning models across the integrated detection framework.

1.2 Research Problem

Modbus/TCP command and response injection attacks are difficult to detect because malicious messages may follow the correct protocol format and contain valid function codes, register addresses and transaction information. Although such messages appear legitimate when analyzed individually, they may be abnormal when compared with the communicating device's role, previous commands, permitted register access, expected values or corresponding request and response.

Existing packet-level and generic flow-based detection approaches may not represent these contextual relationships sufficiently. This creates the risk that an unauthorized command or falsified response will be treated as normal communication. Therefore, this research investigates how Modbus/TCP request response relationships and surrounding command context can be represented as structured features and analyzed using supervised machine learning.

1.2.1 Problem Scope and Boundaries

The proposed component focuses on low-volume manipulations that change the meaning or outcome of Modbus/TCP communication rather than attacks that mainly increase traffic volume. A command-injection event may introduce an unauthorized state change, abnormal parameter value, unexpected

write operation, prohibited register access or inappropriate use of a valid function code. A response-injection event may fabricate or alter the reply received by a supervisory application.

The component determines whether each event is consistent with the communicating device's role, permitted function codes and register ranges, recent operations, command timing, corresponding request or response and expected value transitions. It classifies supported events as normal, command injection or response injection.

Query flooding, ping flooding, TCP SYN flooding and other availability-oriented attacks are outside the scope of the proposed component and belong to the denial-of-service component. Reconnaissance and scanning are also excluded and belong to the reconnaissance-detection component.

The proposed component may use a Modbus transaction identifier to associate a request with its corresponding response. However, it does not detect replay attacks. RP4 is responsible for transaction-ID reuse, duplicated transaction patterns and replay-related timing.

This version is enough for the Problem Scope and Boundaries section. Keep the detailed feature descriptions for the Methodology section to avoid repetition.

1.3 Literature review

1.3.1 Modbus/TCP Security and Command Injection

Modbus/TCP exchanges commands and process measurements among HMIs, SCADA servers, PLCs, sensors and actuators. Standard Modbus/TCP was designed for dependable industrial communication and does not natively provide strong authentication, confidentiality or cryptographic message-integrity protection [1], [2]. Consequently, an attacker with network access may transmit a correctly structured but operationally illegitimate message.

Command injection inserts or modifies a request, such as an unauthorized register-write operation. Response injection fabricates or alters a reply so that the supervisory application receives incorrect information. Because these messages may use valid function codes, addresses and packet structures, detection should consider their relationship with previous commands, device roles, permitted register access and the corresponding request or response.

1.3.2 Command and Response Injection Detection

Gao et al. investigated command and response injection in an experimental SCADA environment and distinguished manipulation of control requests from manipulation of response information [6]. The study demonstrated that protocol and process information can support detection and established the two attacks as separate categories. However, it predates recent Modbus datasets and explainable machine-learning workflows. It did not fully evaluate whether request-response relationships, command history, permissions and value transitions could form a reusable contextual representation.

This work supports the proposed three-class formulation - normal, command injection and response injection and motivates reconstructing request-response events before classification.

1.3.3 Model-Based Modbus/TCP Intrusion Detection

Goldenberg and Wool developed a deterministic model of expected Modbus/TCP behavior for individual HMI–PLC channels [3]. It represented function codes, addresses, message lengths and communication sequences. On production traffic, the authors reported no false alarms for five of seven PLCs for 111 hours and detected genuine operational anomalies [3].

The approach demonstrates the value of protocol-specific behavior and command order. However, exact behavioral models may depend heavily on previously observed patterns. Legitimate operational changes can generate alerts, while an attacker may reuse a permitted operation in a harmful context. The method also does not separately classify command and response injection. The proposed component therefore preserves sequence information but uses Random Forest to learn combinations of command, access, timing, pairing and value evidence.

1.3.4 Machine Learning for Malicious SCADA Communication

Beaver, Borges-Hink and Buckner compared supervised machine-learning classifiers using labelled SCADA network and protocol attributes [8]. Their findings support supervised classification when suitable labelled features are available. A key strength is the comparative evaluation of multiple algorithms on structured industrial-control data.

However, the study focused on broader malicious-communication detection rather than separating command injection from response injection. Individual records may also omit what happened before an event or whether a response agrees with its request. The proposed research retains supervised classification but adds previous-command, request-response, register-access and value-change context.

1.3.5 Industrial-Control Traffic Datasets

Morris and Gao introduced industrial-control traffic datasets for intrusion-detection research [5]. The MSU New Gas Pipeline dataset contains explicit normal and attack labels. For this research, normal records remain normal; naïve and complex malicious response injections are mapped to response injection; and malicious state, parameter and function-code injection are mapped to command injection. Denial-of-service and reconnaissance records are excluded from component training.

The dataset provides explicit injection categories, but it is simulated, relatively old and lacks complete transaction information in some structured records. It therefore cannot support every request–response feature and is used mainly for secondary or external validation with a reduced common feature set.

1.3.6 CICModbusDataset2023

CICModbusDataset2023 provides benign and attack Modbus/TCP PCAP captures with timestamped attack information [7], [12]. Its scenarios include false-data injection, delayed responses, brute-force writes, reconnaissance, query flooding and replay-related activity. Raw PCAP data preserve timestamps, endpoints, traffic direction, transaction and unit identifiers, function codes, addresses, values and potential request–response relationships.

Its main advantage is support for packet parsing, event ordering and transaction reconstruction. However, PCAP packets do not contain a directly usable three-class label. Attack logs must be correlated with traffic using timestamps, scenarios, targets and transaction identifiers where available. Packet direction and request-response position must then be verified. Manipulated requests may be mapped to command injections, while fabricated or manipulated replies may be mapped to response injections. Ambiguous matches are excluded and documented. Query flooding, reconnaissance and replay remain assigned to the other research components.

1.3.7 Random Forest for the Proposed Component

Random Forest combines predictions from multiple decision trees trained using bootstrapped samples and randomly selected feature subsets [9]. It is suitable because the proposed inputs are structured variables describing function codes, previous commands, addresses, read/write transitions, timing, permissions, pair consistency and value changes.

The model can learn nonlinear feature interactions, support multiclass classification, use class weighting and generate class probabilities. Feature importance or a tree-compatible explanation method can identify influential contextual evidence. Nevertheless, Random Forest does not remember sequences automatically. Command history must first be converted into lag, sliding-window, pair and transition features. Therefore, the novelty is the contextual Modbus representation rather than the Random Forest algorithm.

1.3.8 Critical Synthesis of the Literature

The literature confirms that protocol-specific modelling is more appropriate for Modbus environments than relying only on generic network-flow statistics. Command-injection research establishes the attack categories [6], deterministic modelling demonstrates the value of sequence behavior [3], and supervised studies support classification using labelled SCADA data [8].

However, existing work does not sufficiently combine request-response reconstruction, surrounding command behavior, access context and value transitions in an explainable supervised representation that distinguishes normal, command-injection and response-injection events. Dataset compatibility, leakage-resistant partitioning and per-class explanation also remain insufficiently addressed.

These findings motivate Modbus-aware parsing, chronological event preservation, request-response reconstruction, contextual feature engineering, three-class classification, an isolated-event Random Forest baseline, feature-group ablation, scenario- or time-based data splits and per-class evaluation.

1.3.9 Existing Systems and Products

Snort Modbus Preprocessor

Snort decodes Modbus traffic and allows rules to examine function codes, unit identifiers and message data. It also alerts incorrect lengths, non-zero protocol identifiers and reserved function codes [13]. Its protocol-aware rules provide fast detection of known conditions, including operations from

unauthorized hosts. However, a valid-looking injected message may not match a predefined rule, and Snort does not inherently learn contextual relationships from labelled injection data.

Suricata

Suricata provides Modbus-aware application-layer rule keywords for protocol decoding and signature detection [14]. It integrates with wider network-monitoring environments, but performance depends on analyst-defined rules and selected fields. It does not independently provide three-class context-aware Random Forest classification.

Nozomi Networks Guardian

Guardian provides OT asset discovery, network visualization, continuous monitoring and anomaly-based detection. It builds a behavioral baseline to identify suspicious communication, unwanted operations and unusual process values [15]. Its strengths include broad visibility and operational dashboards. However, public documentation does not disclose enough features, training-data or per-class performance detail to reproduce command/response-injection detection academically.

Claroty Continuous Threat Detection

Claroty CTD provides asset inventory, behavioral profiling and multiple threat-detection engines. It profiles assets, communication and processes and produces contextualized alert timelines [16]. Although suitable for industrial security teams, its detailed Modbus injection features, model configuration and reproducible class-level results are not publicly exposed.

1.3.10 Limitations Addressed by the Proposed Component

The proposed component does not attempt to replace full commercial OT-security platforms. Instead, it addresses a specific and experimentally measurable limitation: detecting protocol-valid command and response manipulation using reproducible contextual evidence.

It will address limitations in existing research and systems by:

- Reconstructing requests and responses where transaction information permits.
- Preserving chronological command order.
- Separating command injection and response injection.
- Combining protocol, access, timing, pair-consistency and value context.
- Comparing the context-aware model with an isolated-event Random Forest.
- Removing feature groups through ablation tests to measure their contributions.
- Using capture-, scenario- or time-based splits to reduce data leakage.
- Reporting per-class recall, macro F1, false-positive rate and latency.
- Providing a class probability and short explanation for each completed prediction.
- Publishing the feature definitions, label mapping and experimental procedure for reproducibility.

The specific novelty is therefore not the use of Random Forest alone. It is the construction and validation of an explainable, context-aware Modbus event representation for distinguishing normal communication, command injection and response injection.

1.4 Research Gap

Existing studies demonstrate that Modbus-specific fields and communication patterns are useful for identifying abnormal industrial-control traffic. Gao et al. examined command and response injection attacks and established their importance in SCADA security [6]. Goldenberg and Wool modelled expected Modbus/TCP communication sequences using a deterministic approach [3], while Beaver et al. evaluated supervised machine-learning algorithms using structured SCADA traffic features [8]. Existing systems such as Snort and Suricata also inspect Modbus fields using predefined rules [13], [14], whereas commercial platforms such as Nozomi Guardian and Claroty CTD provide behavioral monitoring and contextual alerts [15], [16].

However, the existing approaches remain inadequate for this individual research problem. Signature-based systems mainly identify known function codes, malformed messages or predefined conditions. Deterministic sequence models can become dependent on previously observed communication patterns. General machine-learning methods commonly analyze individual records or generic network characteristics without explicitly representing the relationship between a request, its response and the surrounding commands. Commercial platforms do not publicly disclose sufficiently detailed datasets, feature definitions, model configurations or per-class results to support reproducible evaluation of command and response injection detection.

Consequently, it remains insufficiently investigated whether combining the following information can reliably separate normal Modbus/TCP communication from command and response injection:

- Current and previous command relationships
- Request-response consistency
- Source and destination roles
- Permitted function and register access
- Command timing and frequency
- Register-value transitions
- Recent read/write behavior

Another limitation concerns dataset preparation. CICModbusDataset2023 provides raw PCAP captures and attack logs, but false-data-injection records cannot automatically be treated as command or response injection. The traffic direction and request-response position must be verified before assigning one of these labels. In contrast, the MSU Gas Pipeline dataset contains explicit command- and response-injection categories but does not provide every transaction field required for complete request-response reconstruction. Therefore, a reproducible label-mapping and common-feature strategy across these datasets has not been sufficiently evaluated.

The proposed research addresses this gap by reconstructing Modbus request-response events where the source data permits, preserving command order and transforming protocol semantics and surrounding

communication into structured contextual features. A supervised Random Forest will classify each supported event as normal, command injection or response injection. The same classifier using only isolated-event features will be implemented as the baseline, allowing the contribution of contextual information to be measured independently of the selected algorithm.

2. OBJECTIVES

2.1 Main Objective

To design, implement and evaluate a context-aware supervised machine-learning component that uses Modbus/TCP protocol semantics and surrounding command context to classify supported events as normal, command injection or response injection.

2.2 Specific Objectives

1. To critically review existing Modbus/TCP injection-detection methods, datasets, protocol-semantic features and evaluation practices to establish the technical requirements and research gaps.
2. To prepare labelled and chronologically ordered Modbus/TCP data using CICModbusDataset2023 as the primary dataset and reconstruct request–response events where the necessary transaction information is available.
3. To design and implement a reproducible contextual feature set that represents command ordering, device roles, permitted register access, timing behavior, request–response consistency and value transitions.
4. To develop and tune a context-aware Random Forest classifier and an isolated-event Random Forest baseline for classifying events as normal, command injection or response injection.
5. To evaluate the proposed model using leakage-resistant scenario-, capture- or time-based data partitions, macro precision, macro recall, macro F1-score, per-class recall, false-positive rate, ROC-AUC, confusion matrices, inference latency and contextual-feature ablation tests.
6. To integrate the predicted class, confidence score and contextual explanation with the shared Modbus attack-detection framework using an agreed event schema.

These objectives are measurable because they produce defined outputs: a literature review, prepared dataset, contextual feature set, two Random Forest configurations, experimental evaluation results and an integrated detection component.

3. RESEARCH DESIGN AND METHODOLOGY

3.1 Research Design

The study follows a quantitative, experimental design with iterative prototype development. A supervised multiclass classifier is appropriate because the selected datasets include attack labels and the required output classes are known. The independent variable is the feature configuration (isolated/static versus context-aware); dependent variables are detection metrics and latency.

3.2 Data Availability and Preparation

Dataset	Planned role	Relevant content	Limitation/mitigation
CICModbusDataset2023	Primary model development	Benign/attack PCAPs and timestamped logs containing attack type, target IP, and transaction ID	Packets require log correlation; verify direction and exclude ambiguous matches.
MSU New Gas Pipeline	Secondary injection validation	Explicit NMRI/CMRI response and MSCI/MPCI/MFCI command categories	No complete MBAP transaction identity; use a reduced common feature profile.
MSU water-storage data	Additional external validation if compatible	Labelled industrial process and attack records	Verify schema and label compatibility before inclusion.
Controlled defensive testbed	Optional gap-filling and integration test	Normal and authorized laboratory scenarios	Use only isolated, authorized infrastructure and documented labels.

Table 1: Data Availability and Preparation

Primary preprocessing steps are: preserve the original CIC PCAP and log files; validate filenames, scenarios, and timestamps; normalize PCAP and log times to a common time zone; extract Modbus/TCP fields; correlate packet events with attack-log records; reconstruct request-response pairs; verify packet direction; assign only evidence-supported Command and Response Injection Detection in Modbus/TCP Using Context-Aware Command Analysis labels; create contextual features; and split by capture, scenario, or time. Flooding, reconnaissance, replay, and unrelated protocol attacks will be excluded from the primary Command and Response Injection Detection in Modbus/TCP Using Context-Aware Command Analysis experiment but may be retained for whole-system integration tests.

3.2.1 CICModbusDataset2023 Labelling Process

CICModbusDataset2023 is labelled through its benign/attack folder organization and timestamped CSV attack logs, not through a label stored inside each PCAP packet. Benign packets will be obtained from documented benign captures. Attack packets will be correlated with log entries using normalized timestamp, target IP address, documented scenario, and transaction ID where available. Function code, direction, request-response relationship, and payload fields will then be inspected before a record is assigned to a Command and Response Injection Detection in Modbus/TCP Using Context-Aware Command Analysis class. Unmatched or ambiguous records will be retained for audit but excluded from supervised training.

- Read the benign and attack PCAP files and their corresponding attack logs.
- Convert packet and log timestamps to the same time zone and preserve millisecond precision.
- Extract network and Modbus/TCP fields with tshark or PyShark.
- Match candidate events using timestamp tolerance, target IP, scenario, and transaction ID.
- Verify whether the event is client-to-server request-side activity or server-to-client response-side activity.
- Assign a Command and Response Injection Detection in Modbus/TCP Using Context-Aware Command Analysis label only when the packet behavior supports that interpretation.
- Record matched, unmatched, ambiguous, and excluded counts in the preprocessing manifest.

CIC scenario	Initial component treatment	Final inclusion rule
Benign	Normal	Retain verified supported Modbus/TCP events.
False data injection	Candidate command or response injection	Use packet directions and request response evidence before mapping.
Brute-force write	Candidate command injection	Retain only verified unauthorized writing events.
Delay response	Response-side anomaly	Analyze separately; do not call forged response without supporting evidence.
Modify length parameters	Protocol manipulation	Exclude from core three-class model unless scope is formally expanded.
Query flooding	DoS Attack detection using protocol-aware feature	Exclude from the proposed component.

	engineering. DoS/flooding	
Reconnaissance	Reconnaissance attack detection using temporal behavior analysis	Exclude from the proposed component.
Baseline replay	Replay attack detection using transaction consistency analysis	Exclude from the proposed component.
Loading payloads / stacked frames	Other protocol manipulation	Exclude from core component experiment.

Table 2: CICModbusDataset2023 Labelling Process

3.2.2 Secondary MSU Label Mapping

Original	Dataset meaning	Component target	Treatment
0	Normal operation	Normal	Retain
1	Naive malicious response injection	Response injection	Retain and remap
2	Complex malicious response injection	Response injection	Retain and remap
3	Malicious state command injection	Command injection	Retain and remap
4	Malicious parameter command injection	Command injection	Retain and remap
5	Malicious function-code injection	Command injection	Retain and remap
6	Denial of service	Outside component	Exclude from primary experiment
7	Reconnaissance	Outside component	Exclude from primary experiment

Table 3: Secondary MSU Label Mapping

Filtering will be applied to a derived working copy. The original dataset, checksums, and original labels will remain unchanged. A preprocessing manifest will record the source file, label map, excluded categories, record counts before and after filtering, and software version. This is an Command and Response Injection Detection in Modbus/TCP Using Context-Aware Command Analysis scope decision, not deleting the original evidence.

3.2.3 Data-Quality Assessment

- Validate expected column names, data types, ranges, units, and label definitions.
- Sort timestamps within each capture or scenario while preserving original row identifiers.
- Distinguish non-applicable fields from genuinely missing data and create missingness indicators where appropriate.
- Measure exact and near duplicates, recognizing that repeated polling may be legitimate industrial behavior.
- Report class counts and proportions before choosing class weights or training-only resampling.
- Check whether timestamps, capture names, constant attack values, or annotations reveal the target label.
- Verify extreme values against protocol ranges and attack descriptions before removing any record.

3.2.4 Preprocessing Tools

Python and Jupyter Notebook will provide the reproducible workflow. SciPy will load ARFF data; pandas and NumPy will support schema checks, filtering, label mapping, ordering, missingness analysis, and feature construction. Scikit-learn will implement preprocessing pipelines, grouped splitting, class weights, model training, probability calibration, and evaluation. Wireshark will support manual packet verification, while tshark or PyShark will extract Modbus/TCP fields from PCAP files. Matplotlib and seaborn will support data-quality and model-evaluation visualizations.

3.3 Contextual Feature Set

Feature group	Examples	Purpose
Current protocol event	Function code, operation type, address, quantity, exception	Represents the event under assessment.
Request–response relation	Pair status, function consistency, transaction/unit match, response delay	Finds missing, inconsistent, or falsified responses where fields exist.
Command sequence	Previous function codes, read → write transition, repeated write count, window diversity	Represents surrounding command order.

Access context	Permitted register/coil, source role, unusual address transition	Represents operational authorization and device behavior.
Process/value context	Value delta, range flag, rate of change, set-point transition	Finds plausible looking but abnormal modifications.
Data-quality context	Missing-field indicators, source-dataset flag	Prevents non-applicable fields from being treated as ordinary zero values.

Table 4: Contextual Feature Set

3.3.1 Feature Construction

Each model row will represent the current event at time t. Fixed-size context will be generated from the previous K events within the same session, device pair, or valid dataset grouping. Candidate window sizes of 3, 5, and 10 will be tuned using training and validation data. Lag variables may include previous function codes, addresses, command-response indicators, and time differences. Rolling variables may include write count, response count, distinct-address count, invalid-transition count, average inter-arrival time, and maximum absolute value change.

The primary CIC feature profile will use available transaction IDs, unit IDs, source/destination roles, function codes, addresses, values, directions, timestamps, pair-consistency indicators, response delay, exception flags, and previous-event windows. The secondary MSU profile will use its command-response indicator, chronology, function, address, process variables, CRC rate, and time. External validation will use only the common features available and semantically equivalent in both sources.

3.3.2 Prevention of Target Leakage

CIC attack names, attack-log labels, folder names, capture filenames, scenario identifiers that directly reveal the attack, and post-event annotations will never be supplied as model inputs. They are used only to construct or audit labels and grouped partitions. The MSU binary result, categorized result, and specific result fields are also labels and will not be used as features. Rolling windows will not cross scenario, session, capture, training, validation, or test boundaries.

3.4 Model Selection: Why Random Forest?

Random Forest is selected because the engineered features are predominantly structured tabular variables, the available labelled data are moderate in size, and decision-tree ensembles learn nonlinear interactions without the data and computing demands of deep sequence models. It is robust to mixed feature scales, supports class weighting, provides feature-importance information, and can produce class probabilities for confidence scoring. Context is not learned automatically: it is explicitly encoded through lag, window, pair, timing, permission, and value-transition features. Hyperparameters will be selected through training-only cross-validation.

3.4.1 How Random Forest Works in Command and Response Injection Detection in Modbus/TCP Using Context-Aware Command Analysis

Random Forest trains many decision trees on bootstrapped samples of the training records. At each node, a tree evaluates a random subset of features, reducing correlation among trees. The forest combines class votes or average probabilities. One tree may use a write-function indicator and abnormal value change, while another may use an unusual address transition and short response delay. The ensemble can therefore learn nonlinear combinations of contextual evidence. Random Forest does not remember sequences automatically; lag, window, pair, timing, permission, and value-transition variables must first convert context into a fixed feature vector.

3.4.2 Candidate Training Configuration

Setting	Candidate values	Reason
Trees	200, 400, 600	Tests stability against computational cost.
Maximum depth	None, 10, 20, 30	Controls model complexity.
Minimum leaf size	1, 2, 5, 10	Reduces rules based on very small groups.
Features per split	sqrt, log2, 0.5	Controls tree diversity.
Class weighting	balanced, balanced_subsample, none	Addresses unequal class frequencies.
Context window	3, 5, 10 events	Selects the contextual horizon.
Decision threshold	Validation-selected	Balances attack recall and false-positive rate.

Table 5: Candidate Training Configuration

Hyperparameters will be selected using grouped or time-aware cross-validation on the training partition. The final test set will be evaluated once after the preprocessing, features, and parameters are frozen. Feature scaling is not required for Random Forest, although categorical encoding and missing-value handling are still required.

3.5 Baseline, Training, and Validation

- Baseline: Random Forest using only current event/static packet fields.
- Proposed model: the same Random Forest family plus contextual and protocol-semantic features.
- Optional benchmark: XGBoost or a simple decision tree to check whether conclusions depend on one learner.
- Class imbalance: class weights and training-only resampling if necessary; no resampling of validation/test data.
- Validation: group/session/time-based train-validation-test split; stratified group cross-validation where feasible.
- Metrics: macro precision, macro recall, macro F1, per-class recall, false-positive rate, ROC-AUC, confusion matrix, and inference latency.
- Ablation: remove sequence, pair-consistency, access, and value-transition groups one at a time.
- External validation: evaluate on a compatible second dataset without retraining where schema mapping permits.

3.5.1 Experimental Matrix

Experiment	Representation	Purpose
Isolated-event baseline	Uses only the current event's static packet and protocol fields.	Measures Random Forest performance without surrounding context.
Proposed contextual model	Adds sequence, timing, register-access, request-response consistency and value-transition features to the baseline fields.	Measures the overall improvement provided by contextual analysis.
Sequence-feature ablation	Uses the proposed model without previous command, command-order and sliding-window features.	Determines how command-sequence information affects detection performance.
Pair-consistency ablation	Uses the proposed model without request-response consistency features.	Determines the contribution of request-response reconstruction where the dataset supports transaction pairing.
Value-feature ablation	Use the proposed model without value-transition features.	Determines whether changes in register or process values improve injection detection.
External validation	Applies the frozen common-feature model to a compatible secondary dataset without additional model tuning.	Assesses whether the component generalizes data from a different industrial environment.

Table 6: Experimental Matrix

3.5.2 Evaluation Interpretation

Macro precision, recall, and F1 give equal importance to all three classes. Per-class recall will show whether either injection type is missing. The false-positive rate matters because excessive alerts reduce operator trust. ROC-AUC and precision-recall AUC will supplement threshold-dependent metrics. Detection latency will be measured on documented hardware. Where feasible, grouped bootstrap confidence intervals will quantify uncertainty.

Success will be assessed comparatively rather than through an unsupported accuracy promise. The proposed model should improve macro F1 and injection recall over the baseline without an unacceptable increase in false positives or latency. Statistical significance and practical effect size will both be considered.

3.5.3 Explainability and Error Analysis

Global permutation importance will identify influential feature groups. A local tree-compatible method such as SHAP may be used if its output is stable and computationally practical. Each alert should show the predicted class, probability, current function and address, and a small number of contextual reasons. False positives and false negatives will be reviewed by class, scenario, function code, address range, and window size to identify label problems, missing context, distribution shift, or model limitations.

3.6 Data Security and Ethics

Only public datasets, authorized laboratory traffic, and non-production devices will be used. Raw captures will be stored with restricted access and checksums; unnecessary IP/device identifiers will be pseudonymized before sharing. No unauthorized traffic will be generated against external or operational systems. Dataset licenses and attribution requirements will be followed. Results will report class imbalance, missing data, false positives, and dataset limitations to avoid overstating safety or real-world generalization.

3.6.1 Reproducibility and Governance

The repository will separate immutable raw data, parsed data, processed features, model artefacts, and results. A data dictionary will record every feature, type, source, transformation, unit, and missing-value rule. Each experiment will record dataset checksums, split manifests, random seeds, package versions, hyperparameters, metrics, and model identifiers. Only appropriately licensed and pseudonymized derived data will be shared.

3.6.2 Threats to Validity

Threat	Possible effect	Mitigation
Simulated or legacy data	Limited representation of	Use multiple sources and

	modern plants	restrict claims.
Dataset artefacts	Unrealistically high results	Leakage audit, grouped splits, ablation, external test.
Class imbalance	Minority attacks are missed	Class weights and macro/per-class metrics.
Incomplete transaction fields	Overstated pairing capability	Reduced MSU profile and PCAP validation.
Label noise	Incorrect learning or scoring	Verify logs, definitions, and ambiguous records.
Concept drift	Performance changes over time	Time-based test and future retraining plan.

Table 7: Threats to Validity

3.7 Development Approach

An iterative Agile prototype approach is selected. Each iteration will produce a testable increment: parser and schema, contextual feature builder, baseline model, proposed model, explanation output, and framework integration. Short review cycles are suitable because feature definitions and dataset compatibility must be refined using evidence from experiments and supervisor feedback.

4. HIGH-LEVEL SYSTEM ARCHITECTURE

4.1 System Overview

Command and Response Injection Detection in Modbus/TCP Using Context-Aware Command Analysis receives live Modbus/TCP traffic or stored PCAP/structured records. The parser extracts available protocol fields. A transaction builder matches a request to its response where identifiers exist and otherwise preserves ordered command records. A context builder derives command-order, access, timing, consistency, and value-transition features. The Random Forest outputs one of three classes—normal, command injection, or response injection—together with class probability and an explanation based on important contributing features. The result is sent to the common fusion, correlation, storage, and dashboard services.

4.1.1 Component Inputs and Outputs

Interface	Required fields	Optional fields	Result
PCAP input	Timestamp and raw packet	Capture/scenario ID	Parsed Modbus/TCP events.
Structured input	Time, function, address/operation	Process values and command-response flag	Ordered contextual records.
Shared repository	Event/session ID, timestamp, protocol fields	Role, permission map, transaction ID	Component inference record.
Component output	Event ID, class, confidence, model version	Register, value, evidence	Fusion, storage, dashboard, and audit.

Table 8: Component Inputs and Outputs

4.1.2 Runtime Behavior

During offline development, labelled records pass through schema validation, partitioning, context generation, model fitting, calibration, and evaluation. During inference, labels are absent. The same versioned preprocessing and feature pipeline transforms each completed event or eligible window before prediction. A pending request may wait for a response or timeout. Malformed or incomplete input is logged as a processing-status event rather than silently classified. The component uses the feature profile validated for the relevant input source.

4.2 Required Technical Figures

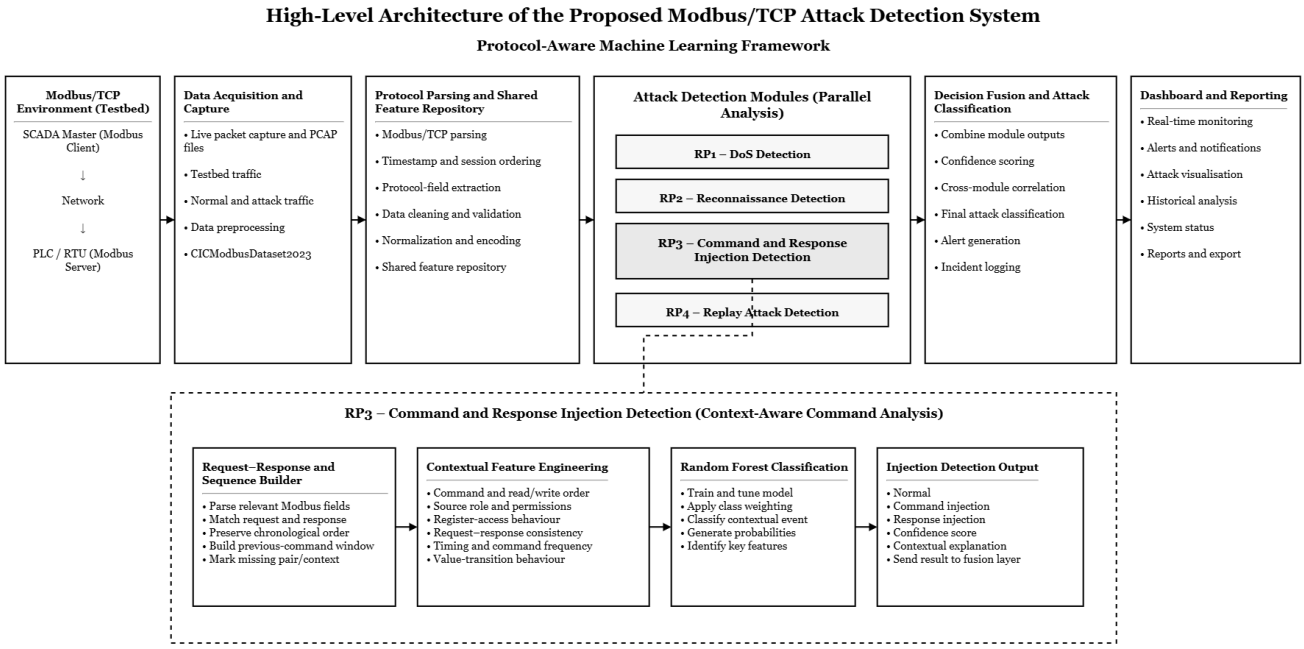


Figure 1: High-Level System Architecture

4.3 Command and Response Injection Detection Processing Pipeline

Figure 2 presents the internal processing pipeline of the Command and Response Injection Detection component. The component receives live Modbus/TCP traffic or stored PCAP data, validates and parses the records, and reconstructs request-response pairs and command sequences. It then generates contextual features representing command order, access behavior, timing, pair consistency and value transitions. A Random Forest classifier uses these features to classify each supported event as normal, command injection or response injection. The component sends the predicted class, confidence score and contextual explanation to the shared decision-fusion and dashboard layers.

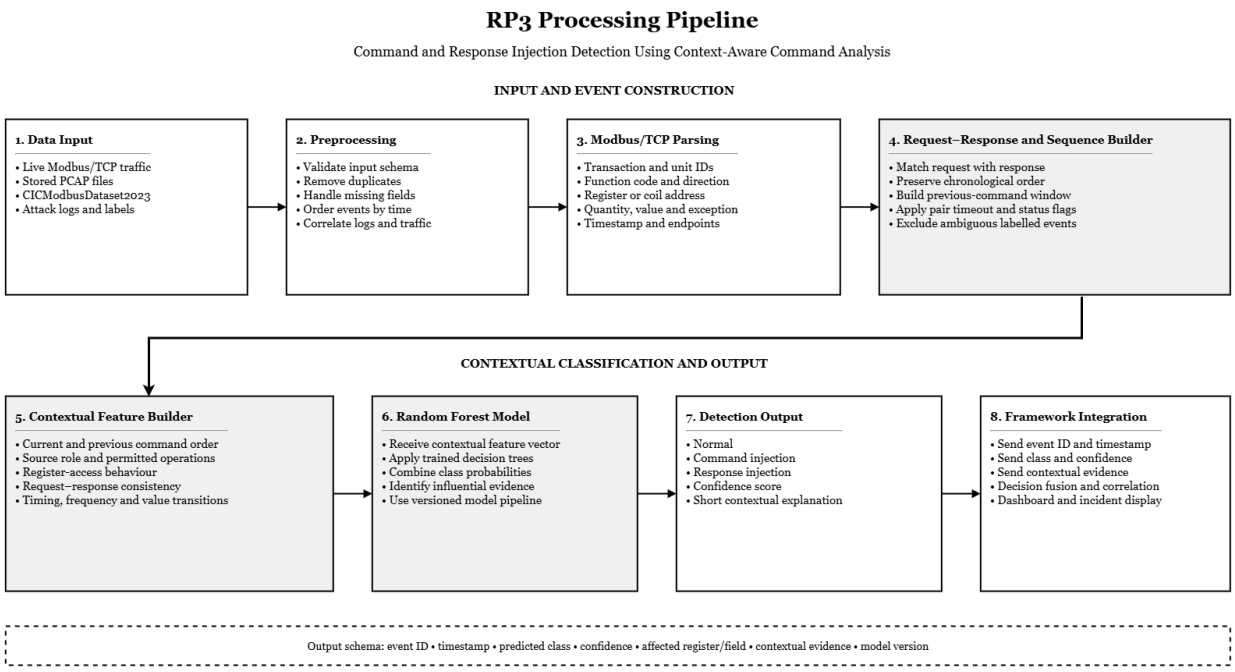


Figure 2: Processing pipeline for context-aware Modbus/TCP command and response injection detection

4.4 Overall Integration and Dependencies

- Inputs from shared services: packet/record schema, timestamps, session identifiers, common preprocessing rules, and dataset labels.
- Command and Response Injection Detection in Modbus/TCP Using Context-Aware Command Analysis-owned processing: pairing/sequence construction, contextual feature engineering, model training, injection classification, and Command and Response Injection Detection in Modbus/TCP Using Context-Aware Command Analysis explanations.
- Outputs to shared services: event identifier, predicted class, confidence score, timestamp, affected asset/register when available, and explanation.
- Dependency on Replay attack detection using transaction consistency analysis is limited: Command and Response Injection Detection in Modbus/TCP Using Context-Aware Command Analysis uses a transaction ID for pairing; RP4 owns duplicate/reuse/replay-timing detection.
- Integration method: versioned JSON/event schema and shared feature repository, followed by API or message-based delivery to decision fusion.

4.4.1 Proposed Result Contract

Field	Example	Purpose
event_id	rp3-000184	Links prediction to the parsed event.
timestamp	ISO 8601 UTC	Supports ordering and correlation.

prediction	command_injection	Provides the component class.
confidence	0.93	Supports fusion and triage.
function_code	16	Identify the operation.
address_or_register	40012	Identifies the affected object.
evidence	unexpected write; large value delta	Provides contextual explanation.
model_version	rp3-rf-1.0	Supports audit and reproduction.
processing_status	complete or insufficient_context	Prevents silent failure.

Table 9: Proposed Result Contract

4.5 Individual Responsibility research Boundary

Activity / Deliverable	My responsibility	Shared responsibility	Other member
Component literature review, gap, objectives	✓		
Injection label mapping and contextual feature definitions	✓		
Component preprocessing, models, evaluation, explanations	✓		
Common capture/parser schema		✓	Team
Shared repository and integration API		✓	Team
DoS detector			Rathnayake H.M.S.R
Reconnaissance			Charith H

detector			
Replay detector			Ilankoon I.M.D.S
Fusion/dashboard integration		✓	Team

Table 10: Individual Responsibility Boundary

5. USER REQUIREMENTS

5.1 Stakeholders and Elicitation Plan

Target stakeholders are OT security analysts, control engineers, network administrators, and system operators. Requirements will be validated through a short, structured interview or questionnaire with a supervisor/domain expert, observation of the testbed workflow, and review of published Modbus security guidance. The proposal does not claim that interviews have already occurred; the questions and evidence log are included in the appendices for completion.

5.1.1 Planned Evidence Collection

- Ask operators which Modbus fields and contextual reasons are meaningful during alert review.
- Ask security analysts what confidence, latency, and false-positive information is required for triage.
- Observe how captures, identifiers, and register maps are accessed in the authorized test environment.
- Present a low-fidelity alert example and test whether the class and explanation are understandable.
- Record stakeholder role, date, consent, finding, resulting requirement, and supervisor confirmation without unnecessary personal data.

5.2 Functional Requirements

- The component shall accept supported PCAP files or structured Modbus records and validate their input schema.
- The component shall extract available Modbus information, including function codes, register or coil addresses, values, timestamps, unit IDs and transaction IDs.
- The component should preserve the chronological order of events and reconstruct request-response pairs when the required transaction information is available.
- The components shall generate the defined contextual features consistently and reproducibly.
- The component shall classify each supported event as normal, command injection or response injection.
- The component shall provide a confidence score and an explanation identifying the contextual evidence that influenced the prediction.
- The component shall store its results using the agreed event schema and transmit them to the unified decision-fusion layer.
- The component shall record the model version, preprocessing version and decision timestamp to support auditing and reproducibility.

5.3 Non-Functional Requirements

- **Performance:** The component shall report its median and 95th-percentile inference latency. The target processing time is no more than 100 milliseconds per prepared event on the documented evaluation machine.
- **Reliability:** The component shall successfully process at least 99% of valid records that follow the supported input schema without pipeline failure.
- **Security:** Access to raw data shall be restricted to authorized users. Nonessential IP addresses and asset identifiers shall be pseudonymized before results or datasets are exported.
- **Reproducibility:** The implementation shall use fixed random seeds and maintain version information for the dataset schema, preprocessing pipeline, contextual-feature code, trained model and software dependencies.
- **Usability:** The system shall clearly display the predicted class, confidence score, event timestamp, affected register or field and a short explanation of the contextual evidence.
- **Maintainability:** The implementation shall use separate modules for packet parsing, log correlation, contextual-feature construction, model training, evaluation and framework integration. Important modules shall be covered by automated tests.
- **Scalability:** The component shall be tested using increasing data volumes. Its throughput, processing time and memory consumption shall be measured, and the maximum tested capacity shall be documented.

5.4 Acceptance Test Summary

- The extracted Modbus fields and generated contextual features will be compared with manually verified sample records to confirm their accuracy.
- The trained Random Forest model will be tested to ensure that it returns only the three defined classes: normal, command injection and response injection.
- Each completed prediction will be checked to confirm that it includes a confidence score and relevant contextual evidence
- The component's output will be validated against the agreed integration schema to ensure that it can be received and processed correctly by the decision-fusion layer.
- The median and 95th-percentile inference latency will be measured using documented evaluation hardware.
- The component will be tested with valid, malformed and incomplete inputs to ensure that it handles errors without causing pipeline failure.
- Raw-data access restrictions will be verified, and exported results will be reviewed to ensure that nonessential asset identifiers have been pseudonymized.
- A fixed experiment will be repeated using the same dataset partitions, random seed, preprocessing configuration and model parameters. The resulting metrics will be compared to confirm that the research is reproducible.

6. COMMERCIALIZATION PLAN

6.1 Target Market, Customer Persona, and Key Stakeholders

- **Target Market:** Small and Medium Enterprises and municipal critical-infrastructure operators running legacy Modbus/TCP networks, such as regional water and wastewater treatment plants, municipal power-distribution environments, food-processing plants, and light-manufacturing facilities. The framework may also be suitable for OT security service providers, system integrators, and industrial training laboratories.
- **Customer Persona:** An OT/SCADA systems engineer, plant automation technician, or security analyst responsible for maintaining network availability, physical-equipment reliability, and control-system monitoring. The user may operate without a dedicated 24-hour Security Operations Centre and therefore require understandable and actionable security alerts.
- **Key Stakeholders:**
    - **Plant Operators and OT Engineers:** Day-to-day users who receive alerts, investigate abnormal Modbus transactions, and require understandable explanations to prevent operational interruptions.
    - **Plant Managers and Asset Owners:** Decision-makers concerned with cost-effectiveness, operational availability, regulatory compliance, and passive deployment without replacing existing PLCs.
    - **IT and OT Security Teams:** Personnel responsible for monitoring network activity, investigating incidents, maintaining access controls, and coordinating security responses.
    - **IT/OT Security Integrators:** External system integrators responsible for deploying network monitoring TAPs or SPAN ports and connecting the framework with existing industrial infrastructure.
    - **Development and Maintenance Team:** The research and engineering team responsible for software updates, model retraining, security patches, documentation, and technical support.

6.2 Value Proposition

The proposed framework delivers an affordable, passive, and software-driven intrusion-detection system designed specifically for Modbus/TCP industrial networks. It analyses four attack categories addressed by the project: denial-of-service, reconnaissance, command and response injection, and replay attacks.

Instead of depending entirely on isolated-packet inspection or fixed detection rules, the framework applies protocol-aware and attack-specific analysis. Each component examines the characteristics associated with its assigned attack category and produces a prediction and confidence score.

The outputs of all four detection components are sent to the common decision-fusion and incident-analysis layer. This layer combines the results and generates a unified alert with an understandable explanation. Therefore, operators can receive one correlated assessment instead of several fragmented alerts.

The system is designed to monitor traffic passively through PCAP files, SPAN ports, network TAPs, or supported traffic sources. It does not require changes to existing PLC programs or direct interference with industrial communication. The framework is also intended to operate on standard industrial computers, edge servers, or virtual machines without requiring specialized proprietary hardware.

6.3 Revenue Model

A flexible hybrid revenue model is proposed to accommodate both operational-expenditure and capital-expenditure purchasing preferences.

- **Tier 1: Annual or Multi-Year Software Subscription**
    - Provides access to the complete four-component detection framework, decision-fusion layer, visualization dashboard, software maintenance, and technical support.
    - May be billed annually or through a multi-year enterprise agreement.
    - Pricing may be calculated according to each monitored SCADA gateway, subnet, site, or number of protected assets.
- **Tier 2: Perpetual License with Annual Maintenance Contract**
    - Provides a one-time software purchase for organizations that require fully on-premises or isolated deployment.
    - May be suitable for facilities that do not permit continuous cloud connectivity or recurring subscription arrangements.
    - An optional Annual Maintenance Contract may cover technical support, security patches, software corrections, compatibility updates, and scheduled model recalibration.
    - The Annual Maintenance Contract may be approximately 18-20% of the initial license cost per year, subject to future market validation.
- **Tier 3: Professional Services and One-Time Deployment**
    - Provides initial on-site or remote deployment.
    - Includes passive network-TAP or SPAN-port configuration.
    - Includes normal-traffic baseline collection and profiling.
    - Includes component configuration and threshold tuning.
    - Includes integration with the customer’s existing monitoring environment.
    - Includes basic operator and administrator training.

The current implementation remains a non-commercial academic prototype. These revenue options represent possible future commercialization approaches.

6.4 Cost Estimation

- **Development and Retraining Compute:** Model development, training, and evaluation can be conducted using standard computers and component-appropriate open-source machine-learning frameworks such as scikit-learn and PyTorch where required. This reduces the requirement for specialized computing hardware.
- **Software Tools:** Python, pandas, NumPy, scikit-learn, Wireshark, TShark, Scapy, PyShark, and other required open-source technologies may be used during development. Therefore, the prototype is not expected to require significant software-licensing costs.
- **Ongoing Maintenance and Patch Releases:** Operational costs may include maintaining a secure software repository, distributing software updates, correcting dependency vulnerabilities, maintaining protocol parsers, and updating the dashboard.
- **Support and Baseline Calibration:** Engineering time may be required during customer onboarding to collect normal traffic, configure the system, establish operational baselines, and tune component-specific thresholds.
- **Documentation and Compliance Packaging:** Costs may arise from preparing and maintaining user manuals, installation instructions, integration guides, model documentation, release notes, and security-verification records.
- **Deployment and Integration:** Additional costs may depend on the customer's network architecture, number of monitored sites, hardware availability, integration requirements, and support level.

6.5 Pricing Strategy

The proposed pricing strategy follows a cost-conscious value-pricing model. Because the framework is designed using open-source technologies and standard computing hardware, it is intended to provide an accessible option for small and medium-sized industrial organizations.

The following prices are preliminary research-stage estimates. They must be validated through stakeholder interviews, competitor analysis, deployment-cost calculations, and willingness-to-pay assessments.

Service Option	Billing Structure	Inclusions and Scope
Complete Framework Subscription	Recurring (LKR 80K / gateway/ year)	All four detection components, decision-fusion layer, visualization dashboard, routine updates, model recalibration, and priority security patches
Perpetual	One-time payment of	On-premises deployment of the complete

Software License	approximately LKR 70K – 100K per installation	detection framework and visualization dashboard
Annual Maintenance Contract	Approximately LKR 60K – 120K per year	Security patches, software corrections, operating-system compatibility updates, model maintenance, and technical support
Onboarding and Tuning Service	One-time payment of approximately LKR 50K – 70K	Network-capture analysis, normal-traffic profiling, component configuration, threshold tuning, integration, and operator training

Table 11: Pricing Strategy

Final prices may vary according to the number of protected sites, gateways, industrial assets, required detection components, deployment type, integration complexity, and support level.

6.6 Maintenance, Versioning, and Patch Updates

To maintain detection accuracy and system stability in continuous industrial environments, the following maintenance lifecycle is proposed:

- **Periodic Version Releases:** Major software versions may be released annually to introduce improved detection models, enhanced integration capabilities, additional reporting functions, and updated dashboard features.
- **Routine Security and Patch Updates:** Maintenance releases may be provided every two months or when required to deliver parser improvements, dependency-security patches, software corrections, and interface refinements.
- **Baseline Recalibration and Model-Drift Maintenance:** Recalibration may be required when a plant updates PLC firmware, introduces new PLCs or RTUs, changes register mappings, modifies device roles, or changes normal polling intervals.
- **Model Version Control:** Each deployed model should have a recorded model version, preprocessing version, feature-set version, training date, and supported input schema.
- **Staged Deployment:** Software and model updates should be evaluated in a testing or staging environment before operational deployment. A rollback option should be available if the update causes unexpected behavior.
- **Passive and Non-Disruptive Operation:** Updates should be deployed using a controlled process designed to minimize disruption to SCADA monitoring and industrial operations.
- **Containerized Deployment:** Container technologies such as Docker may be evaluated to isolate individual components, manage dependencies, and simplify version updates. Their suitability will be confirmed during prototype integration testing.

6.7 Competitive Advantage

- **Comprehensive Multi-Attack Coverage:** The framework integrates specialized components for detecting denial-of-service, reconnaissance, command and response injection, and replay attacks.
- **Protocol-Aware Analysis:** It analyses relevant Modbus/TCP information such as MBAP header fields, function codes, register addresses, transaction relationships, command sequences, timing patterns, and value changes instead of depending only on generic network-flow statistics.
- **Specialized Detection Components:** Each attack category is processed using an analytical method suited to its behavior rather than using one generic detector for every attack.
- **Decision Fusion and Alert Correlation:** Outputs from the four components are combined to provide a unified classification, confidence score, and incident explanation.
- **Low Computational Requirements:** The framework is intended to operate on lightweight industrial computers, edge servers, or virtual machines without requiring expensive specialized hardware.
- **Passive Deployment:** The solution monitors copied network traffic and does not require modification of PLC logic or active interference with the control process.
- **Root-Cause Explainability:** Each alert can identify the evidence contributing to the decision, such as a traffic-rate increase, abnormal scanning sequence, unauthorized register access, inconsistent response, abnormal value transition, or duplicated transaction pattern.
- **Modular Design:** Each detection component can be maintained or updated separately while continuing to communicate through a shared result and integration format.
- **Academic Transparency:** Unlike proprietary systems, the research prototype documents its selected features, models, evaluation methods, assumptions, and limitations.

Any claimed improvements in accuracy, recall, detection latency, or false-positive rate will be supported by experimental results before being presented as verified commercial advantages.

6.8 Intellectual Property Considerations

All intellectual-property matters will be managed according to SLIIT's applicable research and intellectual-property policies. Ownership of the algorithms, protocol-parsing code, trained model files, contextual features, fusion logic, dashboard, diagrams, and documentation must be confirmed with the university, supervisors, and research-team members before commercialization.

Third-party datasets, libraries, and software will not be presented as property of the research team. Public research datasets, including CICModbusDataset2023 and the MSU/ORNL datasets, will be used according to their published access conditions, citation requirements, and applicable licences. Permission for commercial reuse will be verified before using dataset-derived models in a commercial product.

The software-development team will maintain a record of third-party dependencies, versions, and license obligations. A future commercial version may use authorized customer traffic, independently generated testbed data, synthetic data, or datasets with confirmed commercial-use permission.

Customer PCAP files, register mappings, device details, network addresses, baseline profiles, and system configurations will be treated as confidential information. Appropriate controls should include access restrictions, secure storage, data minimization, pseudonymization, retention limits, and secure deletion.

The team may protect its original software implementation, trained models, feature-engineering methods, decision-fusion design, integration interfaces, dashboard, and technical documentation. However, any copyright, patent, licensing, or ownership decision must be reviewed under university policy and applicable legal requirements.

7. BUDGET AND JUSTIFICATION

Category	Justification	Estimated cost
Hardware	A normal laptop or desktop computer is sufficient for data preprocessing and Random Forest training. The proposed component does not require a GPU or specialized hardware.	LKR 0
Software licenses	Python, pandas, NumPy, scikit-learn, Wireshark/TShark, Scapy or PyShark and Jupyter Notebook are free and open-source.	LKR 0
Cloud services	Google Colab can be used as a backup if additional computing resources are required. The free tier is expected to be sufficient.	LKR 0
Human effort	The researcher's own time will be used for the literature review, data preparation, development, testing and documentation. This work is part of the academic research project.	Not separately costed
Dataset access	CICModbusDataset2023 and the MSU New Gas Pipeline dataset are publicly available for academic research, subject to their usage conditions.	LKR 0
Total estimated cost	The research uses existing hardware, public datasets and open-source tools.	**LKR 0**

Table 12: Budget and Justification

7.1 Resource and Project Risk Justification

Random Forest keeps computing requirements suitable for the student's existing computer and reduces dependence on paid cloud services. Raw PCAP processing may require additional storage, but only authorized captures and derived fields need to be kept in the active workspace. If a physical PLC is unavailable, integration can be evaluated in an isolated virtual or Docker-based testbed. No purchase is assumed until available institutional resources are confirmed.

7.2 Project Risk Register

Risk	Likelihood	Impact	Planned response
Raw dataset unavailable or very large	Medium	High	Retain MSU data; acquire approved PCAP subsets; record version.
Schema mismatch	High	Medium	Define common-core and dataset-specific features.
Class imbalance	Medium	High	Class weights, macro metrics, training-only resampling if needed.
Integration delay	Medium	High	The contract early and provide mock output.
Dataset artefact overfitting	Medium	High	Leakage audit, grouped split, ablation, external validation.
Limited stakeholder access	Medium	Medium	Use supervisor/domain expert review and record evidence limits.

Table 13: Project Risk Register

8. WORK BREAKDOWN STRUCTURE (WBS)

WBS	Task	Owner	Deliverable
1.0	Research definition and literature review	Self	Verified gap, problem, objectives
2.0	Dataset acquisition and governance	Whole team	Dataset register, licenses, hashes
2.1	Label/schema mapping	Self	Three-class label map
2.2	Data-quality assessment	Self	Missingness, duplicates, imbalance report
3.0	Component pipeline development	Self	Parser/adaptor and ordered events
3.1	Contextual feature engineering	Self	Versioned feature builder
3.2	Baseline model	Self	Isolated-event RF
3.3	Proposed model	Self	Context-aware RF
4.0	Evaluation	Self	Metrics, ablation, external validation
5.0	Integration	Whole team	Shared event schema and fusion API
6.0	Prototype testing	Whole team	Verified end-to-end alerts
7.0	Documentation and dissemination	Self	Report, presentation, reproducibility package

Table 14: Work Breakdown Structure

9. GANTT CHART

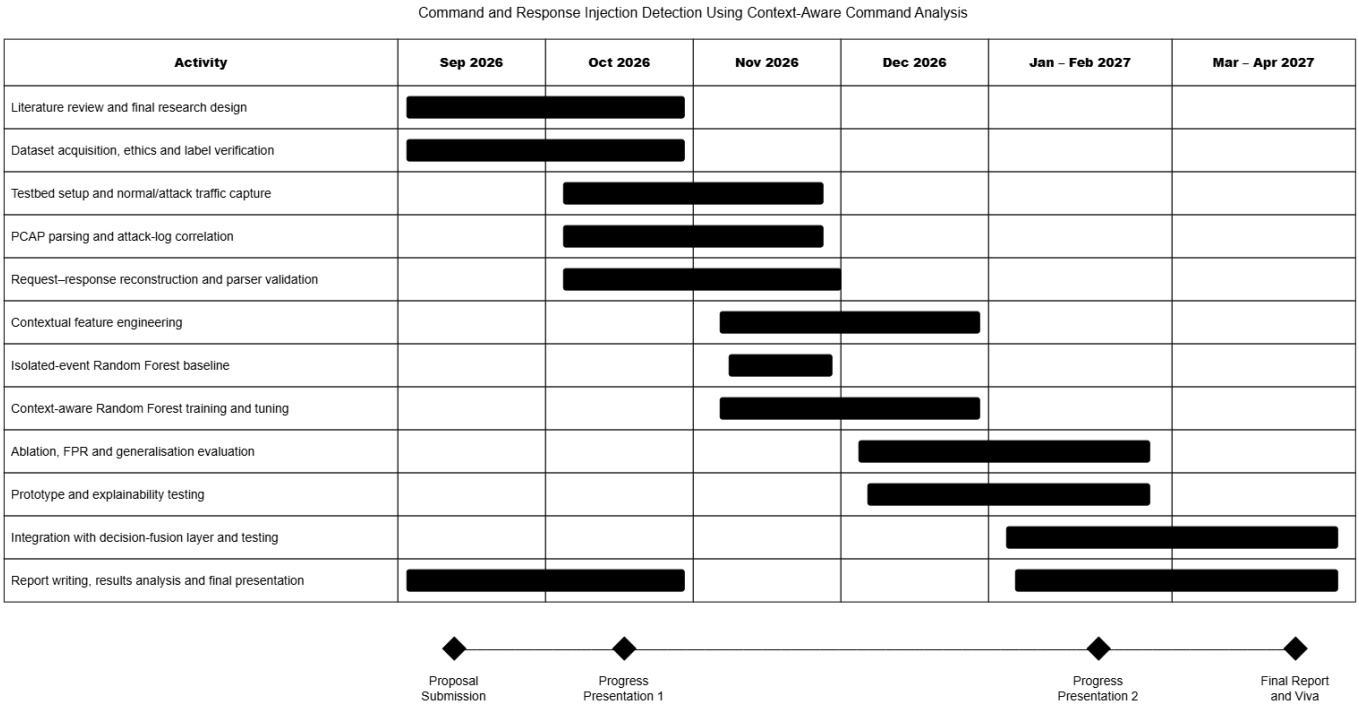


Figure 3: Gantt Chart

10. REFERENCES

[1] Modbus Organization, “MODBUS application protocol specification V1.1b3,” Apr. 2012. https://modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf

[2] K. Stouffer *et al.*, *Guide to Operational Technology (OT) Security*, NIST Special Publication 800-82 Revision 3. Gaithersburg, MD, USA: National Institute of Standards and Technology, Sep. 2023, doi: 10.6028/NIST.SP.800-82r3. https://csrc.nist.gov/pubs/sp/800/82/r3/final.

[3] N. Goldenberg and A. Wool, “Accurate modeling of Modbus/TCP for intrusion detection in SCADA systems,” *International Journal of Critical Infrastructure Protection*, vol. 6, no. 2, pp. 63–75, Jun. 2013, doi: 10.1016/j.ijcip.2013.05.001. https://doi.org/10.1016/j.ijcip.2013.05.001.

[4] T. H. Morris and W. Gao, “Industrial control system cyber attacks,” in *Proceedings of the 1st International Symposium for ICS & SCADA Cyber Security Research*, Leicester, U.K., 2013, pp. 22–29, doi: 10.14236/ewic/ICSCSR2013.3. https://doi.org/10.14236/ewic/ICSCSR2013.3.

[5] T. Morris and W. Gao, “Industrial control system traffic data sets for intrusion detection research,” in *Critical Infrastructure Protection VIII*, IFIP Advances in Information and Communication Technology, vol. 441. Berlin, Germany: Springer, 2014, pp. 65–78, doi: 10.1007/978-3-662-45355-1_5. https://doi.org/10.1007/978-3-662-45355-1_5.

[6] W. Gao, T. Morris, B. Reaves, and D. Richey, “On SCADA control system command and response injection and intrusion detection,” in *2010 eCrime Researchers Summit*, Dallas, TX, USA, 2010, pp. 1–9, doi: 10.1109/ecrime.2010.5706699. https://doi.org/10.1109/ecrime.2010.5706699.

[7] Canadian Institute for Cybersecurity, University of New Brunswick, “CICModbusDataset2023,” 2023. https://www.unb.ca/cic/datasets/modbus-2023.html

[8] J. M. Beaver, R. C. Borges-Hink, and M. A. Buckner, “An evaluation of machine learning methods to detect malicious SCADA communications,” in *2013 12th International Conference on Machine Learning and Applications*, Miami, FL, USA, vol. 2, 2013, pp. 54–59, doi: 10.1109/ICMLA.2013.105. https://doi.org/10.1109/ICMLA.2013.105.

[9] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324. https://doi.org/10.1023/A:1010933404324.

[10] T. Morris, “Industrial control system cyber attack datasets,” University of Alabama in Huntsville. https://sites.google.com/a/uah.edu/tommy-morris-uah/ics-data-sets

[11] T. H. Morris, Z. Thornton, and I. Turnipseed, “Industrial control system simulation and data logging for intrusion detection system research,” in *Proceedings of the 7th Annual Southeastern Cyber Security Summit*, Huntsville, AL, USA, Jun. 2015. [Online]. Available through the University of Alabama in Huntsville dataset repository: https://sites.google.com/a/uah.edu/tommy-morris-uah/ics-data-sets

[12] K. Boakye-Boateng, A. A. Ghorbani, and A. H. Lashkari, “Securing substations with trust, risk posture, and multi-agent systems: A comprehensive approach,” in *2023 20th Annual International Conference on Privacy, Security and Trust (PST)*, Copenhagen, Denmark, 2023, pp. 1–12, doi: 10.1109/PST58708.2023.10320154. https://doi.org/10.1109/PST58708.2023.10320154.

[13] Cisco, “Snort Modbus preprocessor,” *Snort Documentation*. https://www.snort.org/document/readme-modbus

[14] Open Information Security Foundation, “Modbus keyword,” *Suricata User Guide*. https://docs.suricata.io/en/suricata-8.0.4/rules/modbus-keyword.html

[15] Nozomi Networks, “Guardian overview,” *Guardian Technical Documentation*. https://technicaldocs.nozominetworks.com/guardian/latest/products/guardian/topics/intro/c_guardian.html.

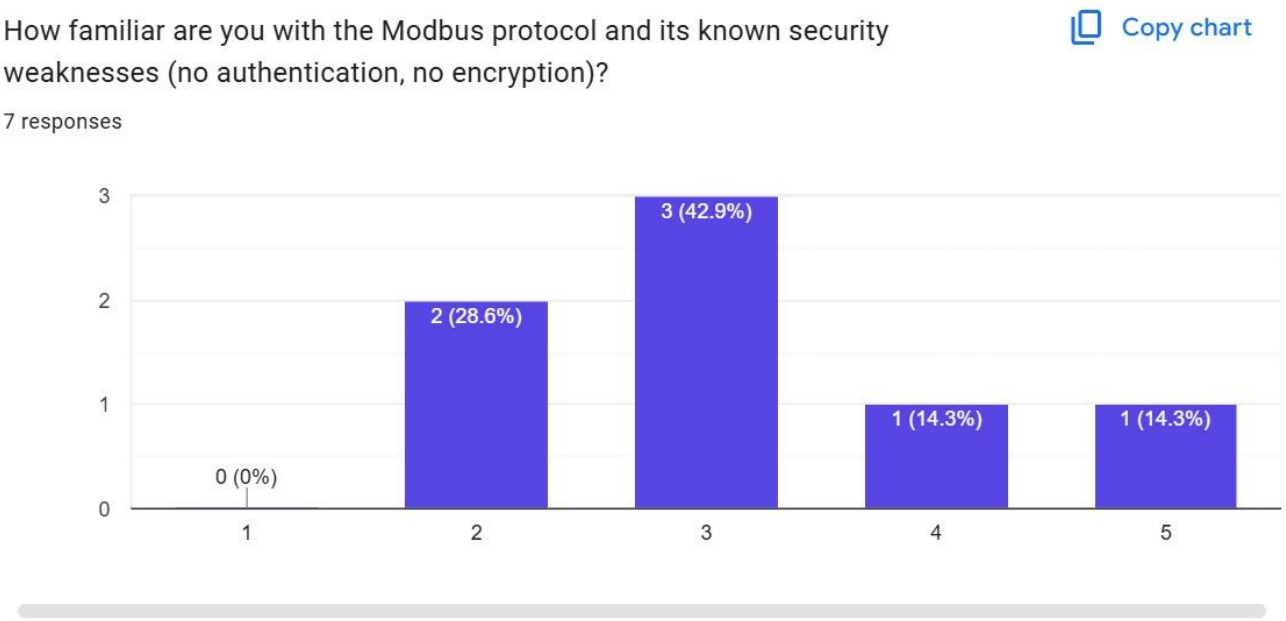
[16] Claroty, “Continuous Threat Detection,” *Claroty Platform*. https://www.claroty.com/commercial-cybersecurity/ctd

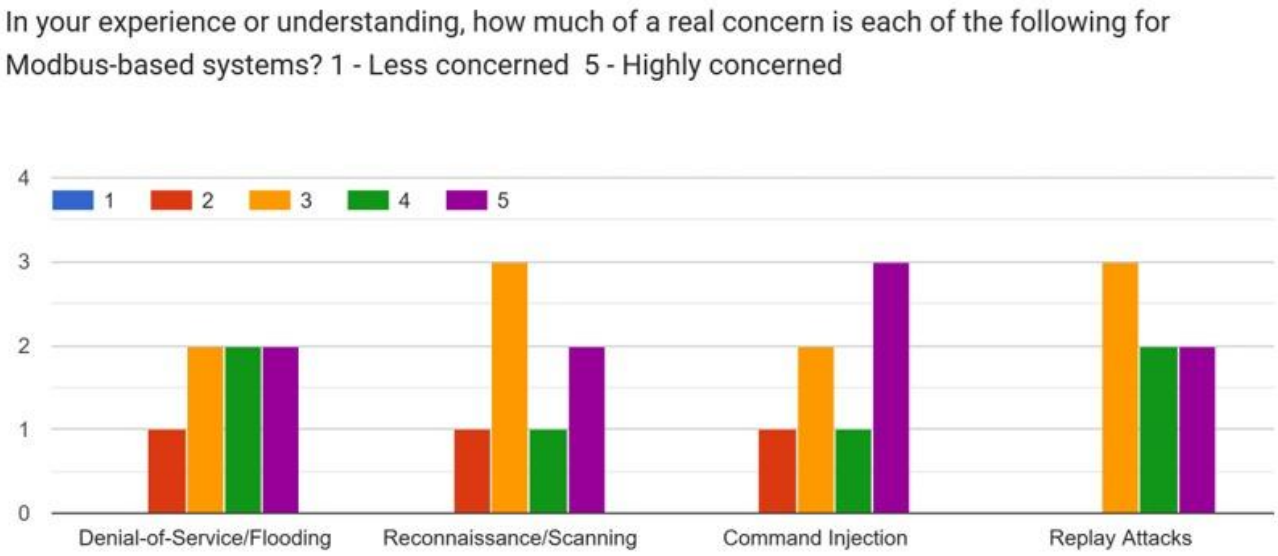
11. APPENDICES

Appendix 1 - Preliminary Dataset and Experimental Plan

- Primary labels: normal; command injection (MSCI, MPCI, MFCI); response injection (NMRI, CMRI).
- Out-of-scope for the primary Command and Response Injection Detection in Modbus/TCP Using Context-Aware Command Analysis model: denial-of-service and reconnaissance; retain only for broader integration tests.
- Unit of analysis: one contextual event representing a current command/response with prior-window and pair-derived attributes.
- Leakage control: group by scenario/capture and keep related records in one partition.
- Reproducibility: save label map, split manifest, schema, feature version, model parameters, and environment file.

Appendix 2 – Survey Evidence





Appendix 3 - Mandatory Proposal Evidence and Decision Log

Date	Claim / Problem	Evidence collected	Alternatives considered	Decision made	Supervisor discussion
Sep 2026	Valid-looking injections may evade isolated-packet detection	Literature and protocol review	Rules only; deep sequence model; tree ensemble	Evaluate explicit contextual features with RF	Pending
Sep 2026	needs labelled command/response classes	MSU dataset scheme and labels	Unlabelled anomaly detection	Use supervised three-class design	Pending
Sep 2026	Primary data lacks complete MBAP pairing fields	Schema audit	Claim full pairing; add raw PCAP	Use ordered context and added PCAP validation	Pending
Sep 2026	Need to prove feature contribution	Experimental design review	Different models only	Same RF baseline with/without context plus ablation	Pending

Appendix 4 - AI Use Disclosure

AI tool	Purpose of use	What was assisted	How output was verified	Final student contribution
ChatGPT	Structuring and language support	Drafting proposal text from the supplied template and RP3 notes	Students check all claims against sources, validate references and data, obtain supervisor approval, and redraw technical diagrams	Students own research decisions, experiments, code, interpretation, editing, presentation, and defense